Guía Semana 7: Base de Datos Local Offline-First con Drizzle ORM y SQLite

Sep 28, 2026 · @AntonioJGL

Esta semana construirás desde cero una app offline-first con SQLite y Drizzle ORM, y luego harás que las fotos de GeoCam sobrevivan al cierre de la app.

Objetivos y preparación

Al terminar esta guía podrás:

1. Elegir la estrategia de almacenamiento local adecuada (AsyncStorage, SecureStore o SQLite) según el tipo de dato.
2. Configurar `expo-sqlite` con Drizzle ORM en un proyecto Expo nuevo, incluyendo migraciones generadas.
3. Modelar esquemas type-safe con relaciones y ejecutar un CRUD completo con consultas reactivas.
4. Encapsular el acceso a datos en una capa de repositorio desacoplada de la UI.

Antes de clase

- [ ] Repasar SQL básico: `CREATE TABLE`, `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `JOIN` y `GROUP BY`.
- [ ] Leer el quickstart de Drizzle para Expo (enlace al final).
- [ ] Tener la GeoCam de la Semana 6 funcionando en tu repositorio.

Preguntas de repaso

Intenta responderlas antes de clase; las discutiremos al inicio.

1. ¿Qué pasa con las fotos de GeoCam si se cierra la app?
2. ¿Qué diferencia hay entre `WHERE` y `HAVING`?
3. ¿Qué es una llave foránea y qué protege?
4. ¿Por qué no se debe guardar dinero en una columna de punto flotante?
5. ¿Dónde guardarías un token de sesión: AsyncStorage o SecureStore?

Conceptos clave

La idea central: los datos estructurados de la app viven en SQLite, se acceden con tipos seguros a través de Drizzle y la UI nunca habla directamente con la base de datos.

Estrategias de almacenamiento local

Opción	Modelo	Ideal para	No usar para
AsyncStorage	Clave-valor, texto, asíncrono	Preferencias, tema, última pestaña	Listas grandes, búsquedas, relaciones
SecureStore	Clave-valor cifrado (Keychain / Keystore)	Tokens, credenciales (Semana 8)	Datos voluminosos (tiene límite de tamaño)
SQLite (`expo-sqlite`)	Base relacional en un archivo `.db`	Registros, filtros, totales, relaciones	Secretos sin cifrar
Sistema de archivos (`expo-file-system`)	Archivos binarios	Fotos, PDFs, audio	Metadatos consultables

Regla práctica: la foto va al sistema de archivos y su ruta, coordenadas y fecha van a SQLite.

Offline-first

Una app offline-first trata la base local como fuente de verdad: escribe primero en el dispositivo y la UI lee de ahí. La red, cuando exista, sincroniza después (Semanas 8 y 9). Beneficios: respuesta instantánea, funciona sin señal y no pierde datos. En zonas rurales o con cobertura intermitente esto no es un lujo: es la diferencia entre una app usable y una inútil.

SQLite y los ORM type-safe

SQLite es una base de datos completa en un solo archivo, sin servidor, incluida en Android e iOS. `expo-sqlite` expone una API asíncrona y síncrona (`openDatabaseSync`) y funciona en Expo Go.

Escribir SQL a mano en strings tiene tres problemas: errores de tipeo que solo aparecen en ejecución, resultados sin tipo (`any`) y riesgo de inyección si se concatenan valores. Un ORM type-safe resuelve los tres: el esquema se define en TypeScript y los tipos de lectura e inserción se infieren solos.

Por qué Drizzle en este curso:

- Su sintaxis se parece a SQL (`select().from().where()`), así que lo que sabes de SQL se transfiere.
- No agrega un motor pesado en ejecución: genera SQL directo.
- Tiene driver oficial para `expo-sqlite` , migraciones y consultas reactivas (`useLiveQuery`).

```
// El esquema ES la fuente de los tipos
export type Expense = typeof expenses.$inferSelect;    // lo que se lee
export type NewExpense = typeof expenses.$inferInsert; // lo que se inserta
```

Migraciones

Una migración es un archivo SQL versionado que lleva la base de un esquema al siguiente. En el celular del usuario ya existe una base con datos; no se puede borrar y crear de nuevo en cada versión.

1. Modificas `db/schema.ts` .
2. Ejecutas `npx drizzle-kit generate` , que crea un nuevo `.sql` en `drizzle/` y actualiza `drizzle/migrations.js` .
3. Al abrir la app, `useMigrations` aplica solo las migraciones pendientes y registra cuáles ya corrió.

Reglas: nunca edites a mano una migración ya distribuida, versiona la carpeta `drizzle/` en Git y no uses `drizzle-kit push` (sirve para bases de servidor, no para la base dentro del celular).

Consultas reactivas y arquitectura por capas

`useLiveQuery` vuelve a ejecutar una consulta cuando cambia la tabla principal que consulta, así la lista se actualiza sola después de un `insert` o `delete` . Requiere abrir la base con `enableChangeListener: true` .

El flujo de datos del curso tiene cinco capas, cada una hablando solo con la siguiente:

1. **Pantalla** (`app/`): muestra datos y captura eventos.
2. **Hook** (`hooks/useExpenses.ts`): une consultas reactivas con acciones.
3. **Repositorio** (`db/repositories/`): única capa que conoce Drizzle; expone funciones como `create` , `update` , `remove` .
4. **Drizzle + esquema** (`db/schema.ts` , `db/client.ts`): tipos, SQL generado y conexión.
5. **expo-sqlite**: el archivo `.db` en el dispositivo.

Si en la Semana 8 los datos pasan a Supabase, solo cambia el repositorio: pantallas y hooks quedan intactos.

Ejemplo de clase desde cero: Billetera Offline

Construirás, desde un proyecto vacío, una app de gastos personales con dos tablas relacionadas, CRUD completo, totales por categoría y lista reactiva. Sigue cada paso en tu computador.

Este dominio obliga a practicar lo que un CRUD de una sola tabla no enseña: llave foránea, `JOIN`, `GROUP BY` con `SUM` y dinero como entero. Para mantener el foco en los datos, el ejemplo usa `StyleSheet`; en tu proyecto puedes seguir usando NativeWind.

Paso 1. Crear el proyecto

```
npx create-expo-app@latest billetera-offline
cd billetera-offline
npm run reset-project   # mueve el ejemplo a app-example/ y deja app/ vacío
```

Paso 2. Dependencias y configuración

```
npx expo install expo-sqlite
npm install drizzle-orm
npm install -D drizzle-kit babel-plugin-inline-import
npx expo customize metro.config.js
```

`drizzle.config.ts` en la raíz. El campo `driver: 'expo'` es obligatorio para que se genere `migrations.js`:

```
import type { Config } from 'drizzle-kit';

export default {
  schema: './db/schema.ts',
  out: './drizzle',
  dialect: 'sqlite',
  driver: 'expo',
} satisfies Config;
```

`babel.config.js`, para que las migraciones `.sql` se puedan importar como texto:

```
module.exports = function (api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo'],
    plugins: [['inline-import', { extensions: ['.sql'] }]],
  };
};
```

`metro.config.js`, para que Metro reconozca la extensión `.sql`:

```
const { getDefaultConfig } = require('expo/metro-config');

const config = getDefaultConfig(__dirname);
config.resolver.sourceExts.push('sql');

module.exports = config;
```

Después de tocar Babel o Metro, reinicia siempre con caché limpia: `npx expo start -c`.

Paso 3. El esquema

```
// db/schema.ts
import { sqliteTable, integer, text } from 'drizzle-orm/sqlite-core';

export const categories = sqliteTable('categories', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  name: text('name').notNull().unique(),
  color: text('color').notNull(),
});

export const expenses = sqliteTable('expenses', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  description: text('description').notNull(),
  amount: integer('amount').notNull(), // pesos enteros, nunca float
  categoryId: integer('category_id')
    .notNull()
    .references(() => categories.id, { onDelete: 'restrict' }),
  spentAt: integer('spent_at', { mode: 'timestamp_ms' })
    .notNull()
    .$defaultFn(() => new Date()),
});

export type Category = typeof categories.$inferSelect;
export type Expense = typeof expenses.$inferSelect;
export type NewExpense = typeof expenses.$inferInsert;
```

Fíjate en tres detalles: `mode: 'timestamp_ms'` hace que Drizzle convierta automáticamente entre `Date` y número; `onDelete: 'restrict'` impide borrar una categoría con gastos; y `$inferInsert` deja `id` y `spentAt` como opcionales porque tienen valor por defecto.

Paso 4. Generar la migración y crear el cliente

```
npx drizzle-kit generate
```

Abre el `.sql` generado en `drizzle/`: es el `CREATE TABLE` que habrías escrito a mano.

```
// db/client.ts
import { drizzle } from 'drizzle-orm/expo-sqlite';
import { openDatabaseSync } from 'expo-sqlite';
import * as schema from './schema';

export const expoDb = openDatabaseSync('billetera.db', {
enableChangeListener: true });

// SQLite no aplica llaves foráneas a menos que se active en cada conexión
expoDb.execSync('PRAGMA foreign_keys = ON;');

export const db = drizzle(expoDb, { schema });
```

```
// db/seed.ts
import { db } from './client';
import { categories } from './schema';

const DEFAULT_CATEGORIES = [
  { name: 'Comida', color: '#F97316' },
  { name: 'Transporte', color: '#3B82F6' },
  { name: 'Estudio', color: '#8B5CF6' },
  { name: 'Ocio', color: '#10B981' },
];

export async function seedCategories() {
  // name es unique: si ya existen, no se duplican
  await
db.insert(categories).values(DEFAULT_CATEGORIES).onConflictDoNothing();
}
```

Paso 5. Layout raíz con migraciones

```
// app/_layout.tsx
import { useEffect, useState } from 'react';
import { ActivityIndicator, Text, View } from 'react-native';
import { Stack } from 'expo-router';
import { useMigrations } from 'drizzle-orm/expo-sqlite/migrator';
import migrations from '@/drizzle/migrations';
import { db } from '@/db/client';
import { seedCategories } from '@/db/seed';

export default function RootLayout() {
  const { success, error } = useMigrations(db, migrations);
  const [seeded, setSeeded] = useState(false);

  useEffect(() => {
    if (success) seedCategories().then(() => setSeeded(true));
  }, [success]);

  if (error) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', padding: 24 }}>
        <Text>Error al preparar la base de datos: {error.message}</Text>
      </View>
    );
  }

  if (!success || !seeded) {
    return <ActivityIndicator style={{ flex: 1 }} />;
  }

  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: 'Billetera Offline' }} />
      <Stack.Screen name="gasto" options={{ presentation: 'modal', title:
'Gasto' }} />
    </Stack>
  );
}
```

Ninguna pantalla se monta antes de que la base esté migrada: así ninguna consulta corre contra tablas inexistentes.

Paso 6. Capa de repositorio

```
// db/repositories/expenses.ts
import { desc, eq, sql } from 'drizzle-orm';
import { db } from '@/db/client';
import { categories, expenses, type NewExpense } from '@/db/schema';

type ExpenseInput = Omit<NewExpense, 'id'>;

export const expensesRepo = {
  listQuery() {
    return db
      .select({
        id: expenses.id,
        description: expenses.description,
        amount: expenses.amount,
        spentAt: expenses.spentAt,
        categoryName: categories.name,
        categoryColor: categories.color,
      })
      .from(expenses)
      .innerJoin(categories, eq(expenses.categoryId, categories.id))
      .orderBy(desc(expenses.spentAt));
  },

  totalsQuery() {
    return db
      .select({
        categoryId: categories.id,
        category: categories.name,
        color: categories.color,
        total: sql<number>`coalesce(sum(${expenses.amount}),
0)`.mapWith(Number),
      })
      .from(expenses)
      .innerJoin(categories, eq(expenses.categoryId, categories.id))
      .groupBy(categories.id);
  },

  findById(id: number) {
    return db.select().from(expenses).where(eq(expenses.id, id)).get();
  },

  create(data: ExpenseInput) {
    return db.insert(expenses).values(data);
  },
```

```
  update(id: number, data: Partial<ExpenseInput>) {
    return db.update(expenses).set(data).where(eq(expenses.id, id));
  },

  remove(id: number) {
    return db.delete(expenses).where(eq(expenses.id, id));
  },
};
```

```
// db/repositories/categories.ts
import { asc } from 'drizzle-orm';
import { db } from '@/db/client';
import { categories } from '@/db/schema';

export const categoriesRepo = {
  listQuery() {
    return db.select().from(categories).orderBy(asc(categories.name));
  },
};
```

Las funciones `...Query()` devuelven la consulta sin ejecutarla, para que el hook la pase a `useLiveQuery`. Las demás se ejecutan con `await`.

Paso 7. Hook que une datos y acciones

```
// hooks/useExpenses.ts
import { useCallback } from 'react';
import { useLiveQuery } from 'drizzle-orm/expo-sqlite';
import { expensesRepo } from '@/db/repositories/expenses';

export function useExpenses() {
  const { data: expenses, error } = useLiveQuery(expensesRepo.listQuery());
  const { data: totals } = useLiveQuery(expensesRepo.totalsQuery());

  const grandTotal = totals.reduce((acc, t) => acc + t.total, 0);
  const remove = useCallback((id: number) => expensesRepo.remove(id), []);

  return { expenses, totals, grandTotal, error, remove };
}
```

```
// utils/money.ts
export const formatCOP = (value: number) =>
  new Intl.NumberFormat('es-CO', {
    style: 'currency',
    currency: 'COP',
    maximumFractionDigits: 0,
  }).format(value);
```

Paso 8. Pantalla de lista

```
// app/index.tsx
import { Alert, FlatList, Pressable, StyleSheet, Text, View } from 'react-
native';
import { Link, router } from 'expo-router';
import { useExpenses } from '@/hooks/useExpenses';
import { formatCOP } from '@/utils/money';

export default function HomeScreen() {
  const { expenses, totals, grandTotal, remove } = useExpenses();

  const confirmDelete = (id: number, description: string) => {
    Alert.alert('Eliminar gasto', `¿Eliminar "${description}"?`, [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Eliminar', style: 'destructive', onPress: () => remove(id) },
    ]);
  };

  return (
    <View style={styles.container}>
      <View style={styles.summary}>
        <Text style={styles.summaryLabel}>Total gastado</Text>
        <Text style={styles.summaryValue}>{formatCOP(grandTotal)}</Text>
        {totals.map((t) => (
          <Text key={t.categoryId} style={{ color: t.color }}>
            {t.category}: {formatCOP(t.total)}
          </Text>
        ))}
      </View>

      <FlatList
        data={expenses}
        keyExtractor={(item) => String(item.id)}
        ListEmptyComponent={<Text style={styles.empty}>Aún no hay gastos.
Agrega el primero.</Text>}
        renderItem={({ item }) => (
```

```
          <Pressable
            style={styles.row}
            onPress={() => router.push({ pathname: '/gasto', params: { id:
String(item.id) } })}
            onLongPress={() => confirmDelete(item.id, item.description)}
          >
            <View style={[styles.dot, { backgroundColor: item.categoryColor
}]} />
            <View style={{ flex: 1 }}>
              <Text style={styles.rowTitle}>{item.description}</Text>
              <Text style={styles.rowSubtitle}>
                {item.categoryName} · {item.spentAt.toLocaleDateString('es-
CO')}
              </Text>
            </View>
            <Text style={styles.rowAmount}>{formatCOP(item.amount)}</Text>
          </Pressable>
        )}
      />

      <Link href="/gasto" asChild>
        <Pressable style={styles.fab} accessibilityLabel="Agregar gasto">
          <Text style={styles.fabText}>+</Text>
        </Pressable>
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  summary: { padding: 20, gap: 4, borderBottomWidth: 1, borderColor: '#eee'
},
  summaryLabel: { color: '#666' },
  summaryValue: { fontSize: 28, fontWeight: '700' },
  empty: { textAlign: 'center', marginTop: 40, color: '#888' },
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 16,
borderBottomWidth: 1, borderColor: '#f2f2f2' },
  dot: { width: 12, height: 12, borderRadius: 6 },
  rowTitle: { fontSize: 16, fontWeight: '600' },
  rowSubtitle: { color: '#777', fontSize: 13 },
  rowAmount: { fontWeight: '700' },
  fab: { position: 'absolute', right: 24, bottom: 40, width: 56, height: 56,
```

```
borderRadius: 28, backgroundColor: '#111', alignItems: 'center',
justifyContent: 'center' },
  fabText: { color: '#fff', fontSize: 28 },
});
```

Paso 9. Formulario para crear y editar

```
// app/gasto.tsx
import { useEffect, useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { router, useLocalSearchParams } from 'expo-router';
import { useLiveQuery } from 'drizzle-orm/expo-sqlite';
import { categoriesRepo } from '@/db/repositories/categories';
import { expensesRepo } from '@/db/repositories/expenses';

export default function GastoScreen() {
  const { id } = useLocalSearchParams<{ id?: string }>();
  const expenseId = id ? Number(id) : null;

  const { data: categories } = useLiveQuery(categoriesRepo.listQuery());
  const [description, setDescription] = useState('');
  const [amount, setAmount] = useState('');
  const [categoryId, setCategoryId] = useState<number | null>(null);
  const [error, setError] = useState<string | null>(null);

  // Modo edición: cargar el gasto existente
  useEffect(() => {
    if (!expenseId) return;
    expensesRepo.findById(expenseId).then((e) => {
      if (!e) return;
      setDescription(e.description);
      setAmount(String(e.amount));
      setCategoryId(e.categoryId);
    });
  }, [expenseId]);

  const save = async () => {
    const value = Number(amount);
    if (!description.trim()) return setError('Escribe una descripción.');
    if (!Number.isInteger(value) || value <= 0) return setError('El monto
debe ser un entero mayor que 0.');
    if (!categoryId) return setError('Elige una categoría.');

    const data = { description: description.trim(), amount: value, categoryId
};
    if (expenseId) await expensesRepo.update(expenseId, data);
```

```
    else await expensesRepo.create(data);
    router.back();
  };

  return (
    <View style={styles.container}>
      <TextInput style={styles.input} placeholder="Descripción" value=
{description} onChangeText={setDescription} />
      <TextInput style={styles.input} placeholder="Monto en pesos"
keyboardType="number-pad" value={amount} onChangeText={setAmount} />

      <View style={styles.chips}>
        {categories.map((c) => (
          <Pressable
            key={c.id}
            onPress={() => setCategoryId(c.id)}
            style={[styles.chip, { borderColor: c.color }, categoryId ===
c.id && { backgroundColor: c.color }]}
          >
            <Text style={categoryId === c.id ? styles.chipTextActive :
undefined}>{c.name}</Text>
          </Pressable>
        ))}
      </View>

      {error && <Text style={styles.error}>{error}</Text>}

      <Pressable style={styles.button} onPress={save}>
        <Text style={styles.buttonText}>{expenseId ? 'Guardar cambios' :
'Agregar gasto'}</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, gap: 12, backgroundColor: '#fff' },
  input: { borderWidth: 1, borderColor: '#ddd', borderRadius: 10, padding:
12, fontSize: 16 },
  chips: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  chip: { borderWidth: 2, borderRadius: 999, paddingHorizontal: 14,
paddingVertical: 6 },
  chipTextActive: { color: '#fff', fontWeight: '600' },
  error: { color: '#DC2626' },
```

```
  button: { backgroundColor: '#111', padding: 14, borderRadius: 10,
alignItems: 'center' },
  buttonText: { color: '#fff', fontWeight: '600' },
});
```

Paso 10. Verifica en tu teléfono

- [ ] Agrega tres gastos y observa cómo la lista y los totales se actualizan solos.
- [ ] Cierra Expo Go por completo, vuelve a abrir y comprueba que los datos siguen ahí.
- [ ] Activa modo avión y verifica que todo funciona igual.
- [ ] Edita un gasto y cámbiale la categoría; confirma que los totales se recalculan.

Taller en clase: GeoCam persistente

En parejas, reemplaza el `GeoPhotosContext` en memoria de la Semana 6 por SQLite con Drizzle, de modo que las fotos y sus coordenadas sobrevivan al cierre de la app. Trabaja sobre el repositorio del Taller Integrador 2 y aplica los pasos 2, 4 y 5 del ejemplo.

Esquema de partida

El tipo `GeoPhoto` de la Semana 6 se traduce así. Las coordenadas son columnas opcionales porque la foto puede no tener ubicación:

```
// db/schema.ts
import { sqliteTable, integer, real, text } from 'drizzle-orm/sqlite-core';

export const photos = sqliteTable('photos', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  uri: text('uri').notNull(),
  latitude: real('latitude'),
  longitude: real('longitude'),
  accuracy: real('accuracy'),
  source: text('source', { enum: ['camera', 'gallery'] }).notNull(),
  createdAt: integer('created_at', { mode: 'timestamp_ms' })
    .notNull()
    .$defaultFn(() => new Date()),
});

export type Photo = typeof photos.$inferSelect;
export type NewPhoto = typeof photos.$inferInsert;
```

`real` es correcto aquí: las coordenadas sí son decimales, a diferencia del dinero.

Requisitos

- [ ] **T1. Configuración.** Instala dependencias, crea `drizzle.config.ts`, `babel.config.js` (fusionando con la configuración existente de NativeWind), `metro.config.js`, y genera la primera migración.
- [ ] **T2. Cliente y migraciones.** Crea `db/client.ts` y protege el layout raíz con `useMigrations`.
- [ ] **T3. Repositorio.** Escribe `db/repositories/photos.ts` con `listQuery`, `withLocationQuery` (solo fotos con latitud no nula, usando `isNotNull`), `create`, `remove` y `clearAll`.
- [ ] **T4. Hook usePhotos.** Expone `photos`, `photosWithLocation`, `addPhoto(input)` y `removePhoto(id)`. `addPhoto` recibe `coords: Coords | null` y lo convierte a columnas planas.
- [ ] **T5. Integración.** GeoCam y la pestaña Mapa usan `usePhotos` en lugar del Context. `useShake` llama a `clearAll` del repositorio.
- [ ] **T6. Verificación.** Toma tres fotos, cierra Expo Go, abre de nuevo y confirma que el mapa conserva los marcadores.

Pistas

- Al fusionar `babel.config.js`, conserva los presets de NativeWind y solo agrega el plugin `inline-import`.
- Para T4, la conversión es: `latitude: coords?.latitude ?? null`, y lo mismo para `longitude` y `accuracy`.
- Si la app muestra "no such table", la migración no se aplicó: revisa que `migrations.js` se importó en el layout y reinicia con `npx expo start -c`.
- La foto de la cámara vive en la carpeta de caché y el sistema puede borrarla. Por ahora se acepta así; el trabajo para la casa lo corrige.

Trabajo para la casa

En parejas, convierte la GeoCam persistente en un módulo CRUD completo con relaciones, una segunda migración y archivos permanentes. Entrega antes de la clase de la Semana 8.

Tareas

- [ ] **C1. Tabla de álbumes.** Crea `albums` (`id`, `name` único, `createdAt`) y agrega a `photos` la columna `albumId` como llave foránea opcional con `onDelete: 'set null'`. Al borrar un álbum, sus fotos quedan sin álbum, no se borran.

- [ ] **C2. Segunda migración.** Agrega a `photos` las columnas `note` (texto opcional) y `favorite` (`integer('favorite', { mode: 'boolean' })` con valor por defecto `false`). Genera la migración y pruébala sobre un celular que ya tenga fotos de la clase: los datos anteriores deben conservarse.
- [ ] **C3. CRUD completo.** Pantalla `app/foto/[id].tsx` para ver una foto, editar su nota, marcarla como favorita, moverla de álbum y eliminarla con confirmación.
- [ ] **C4. Búsqueda y filtros.** En la lista de fotos, un buscador por nota (`like`) y filtros por álbum y por favoritas, usando `useLiveQuery(consulta, [busqueda, albumId, soloFavoritas])` para que se recalcule al cambiar los filtros.
- [ ] **C5. Archivos permanentes.** Al guardar una foto, cópiala de la caché a la carpeta de documentos de la app y guarda en SQLite la nueva ruta. Al eliminar el registro, elimina también el archivo.
- [ ] **C6. Documentación.** `README.md` con el diagrama de tablas (imagen o texto) y un GIF del flujo, más el `AI-LOG.md` de la semana.

Referencia para C5

La API de `expo-file-system` cambió en los SDK recientes: la nueva se basa en las clases `File`, `Directory` y `Paths`, y la anterior quedó en `expo-file-system/legacy`. Verifica en la documentación de tu versión de SDK antes de copiar este esquema:

```
// services/photoFiles.ts
import { Directory, File, Paths } from 'expo-file-system';

const photosDir = new Directory(Paths.document, 'photos');

export function persistPhoto(cacheUri: string): string {
  if (!photosDir.exists) photosDir.create();
  const source = new File(cacheUri);
  const destination = new File(photosDir, `${Date.now()}.jpg`);
  source.copy(destination);
  return destination.uri;
}

export function deletePhotoFile(uri: string) {
  const file = new File(uri);
  if (file.exists) file.delete();
}
```

Esta es una fuente frecuente de alucinaciones: la IA suele mezclar `FileSystem.copyAsync` (API anterior) con las clases nuevas. Si ocurre, repórtalo en el AI-LOG.

Reto opcional

Escribe una tercera migración que cree un índice sobre `photos.created_at` y mide con `console.time` la diferencia al listar 1.000 fotos de prueba insertadas con un script de seed. Es un adelanto de la Semana 11 (rendimiento).

Entregable semanal

Entrega el enlace a tu repositorio de GitHub con un módulo CRUD funcional sobre SQLite y Drizzle: el taller en clase más el trabajo para la casa, con la carpeta `drizzle/` versionada.

Antes de entregar, verifica

- [ ] Hay al menos dos migraciones generadas con `drizzle-kit generate` y ninguna editada a mano.
- [ ] Las fotos de antes de la segunda migración siguen ahí después de aplicarla.
- [ ] Las pantallas no importan nada de Drizzle: todo pasa por hooks y repositorios.
- [ ] Crear, leer, actualizar y borrar funcionan, y los filtros se actualizan solos.
- [ ] Las fotos se guardan en la carpeta de documentos y se borran junto con su registro.
- [ ] `README.md` y `AI-LOG.md` están en la raíz.

Errores de IA que debes detectar

La IA genera	Problema	Corrección
`SQLite.openDatabase('db')` o `db.transaction(tx => tx.executeSql(...))`	API antigua de `expo-sqlite`, retirada	`openDatabaseSync` o `openDatabaseAsync`
`import { drizzle } from 'drizzle-orm/better-sqlite3'`	Driver de Node, no funciona en el celular	`drizzle-orm/expo-sqlite`
`drizzle.config.ts` sin `driver: 'expo'`	No genera `migrations.js`	Agregar `driver: 'expo'`
Falta el plugin `inline-import` o la extensión `sql` en Metro	Error al importar migraciones	Configurar Babel y Metro (Paso 2)
`npx drizzle-kit push`	Pensado para bases de servidor	`npx drizzle-kit generate` + `useMigrations`

La IA genera	Problema	Corrección
Abrir la base sin `enableChangeListener`	`useLiveQuery` no se actualiza	`openDatabaseSync(nombre, { enableChangeListener: true })`
Dinero en `real`	Errores de redondeo	Enteros en la unidad más pequeña
Llaves foráneas sin `PRAGMA foreign_keys = ON`	SQLite no las hace cumplir	Ejecutar el PRAGMA al abrir la conexión
`FileSystem.copyAsync` mezclado con `new File()`	Mezcla de API anterior y nueva	Usar una sola API según el SDK

Ejemplo de AI-LOG.md

```
# Registro de Auditoría de IA (AI-LOG)

**Estudiante(s):** Nombre Apellido, Nombre Apellido
**Semana:** 7
**Proyecto:** GeoCam persistente – Taller Integrador 2

## 1. Prompts Utilizados
- "Crea con Drizzle ORM y expo-sqlite una tabla albums relacionada con
photos,
  de forma que al borrar un álbum las fotos no se eliminen."

## 2. Código Generado vs. Código Modificado
- **¿Qué generó la IA?:** El esquema con references() correcto, pero con
  onDelete: 'cascade', que borraba las fotos junto con el álbum.
- **¿Qué modifiqué/corregí?:** Cambié a onDelete: 'set null', dejé albumId
  opcional y agregué PRAGMA foreign_keys = ON en el cliente, porque sin él
  SQLite ignoraba la regla.

## 3. Alucinaciones o Errores Detectados
- La IA configuró el cliente con `drizzle-orm/better-sqlite3`, que es un
  driver para Node. Lo reemplacé por `drizzle-orm/expo-sqlite` con
  openDatabaseSync según el quickstart oficial de Drizzle para Expo.
```

Documentación de referencia

- Expo SQLite
- Drizzle ORM: quickstart con Expo SQLite
- Drizzle ORM: consultas select

- Expo FileSystem

Para la próxima semana: crea un proyecto en Supabase y lee el quickstart de Expo con React Native. Piensa en esta pregunta: si cambias de celular, ¿dónde quedan tus fotos?